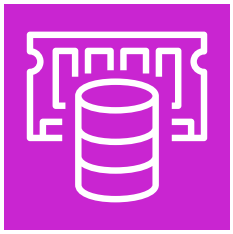
Amazon ElastiCache and Use Cases

Explore Amazon ElastiCache as a fully managed in-memory caching service designed to enhance application speed and scalability. Learn about serverless and cluster deployment options, supported caching engines like Redis, Valkey, and Memcached, and practical use cases such as session management and real-time analytics. Understand how to configure backups and choose the right caching engine to optimize your AWS applications.

Amazon ElastiCache is a fully managed in-memory caching service offered by AWS that improves application performance by retrieving data from fast, managed, in-memory data stores. It eliminates the need for manual infrastructure provisioning, software patching, and complex setup. With built-in scalability, ElastiCache can automatically adjust to workload demands, making it ideal for use cases like real-time analytics, caching, and session management.



Deployment options in ElastiCache

ElastiCache supports two types of deployment formats: serverless caching and self-designed cluster.

Ask

Serverless caching

The serverless option simplifies the deployment and scaling of the cluster according to the application's requirements. It deploys a cache in under a minute and spares the developer from:

- Planning and managing the cache capacity by automatically monitoring and adjusting it to match the application's requirements.
- Managing the cache design by abstracting the underlying infrastructure and providing a simple endpoint for use.

This deployment option is typically recommended when the application's traffic patterns are unpredictable or the developers want to start with the easiest way to deploy a cache.

Self-designed clusters

The self-designed clusters offer a fine-grained control over Valkey, Memcached, or Redis OSS clusters. It allows us to configure:

- The placement of nodes in a cluster: Single AZ or Multi-AZ.
- Number of nodes in the cluster.
- Cluster mode for horizontal scaling and non-cluster mode for vertical scaling.

Self-designed clusters allow us to better control the costs. However, they are only recommended when the capacity requirements can be forecasted.

The diagram below shows a brief comparison between serverless and self-designed clusters:

Ask

Serverless caching vs. self-designed clusters

ElastiCache for Redis, Valkey, and Memcached

ElastiCache supports three powerful caching engines: Valkey, Memcached, and Redis OSS. Developers can switch to these managed solutions in their applications with minimal effort.

ElastiCache for Redis, Valkey, and Memcached

ElastiCache for Redis

Ask

Redis OSS (open-source software) is a widely adopted, versatile in-memory data store that supports complex data structures such as lists, hashes, sets, and

sorted sets. It also includes built-in features like replication, persistence, automatic failover, and Multi-AZ support. Redis is ideal for use cases requiring both performance and resilience.

Consider using Redis in the following scenarios:

- The application requires high availability and durability through replication and persistence.
- We need support for advanced data structures like strings, hashes, lists, sets, sorted sets, and bitmaps.
- Our workloads include real-time features like chat messaging, counters, or tracking user activity.
- We want built-in support for pub/sub patterns in event-driven architectures.
- We need Multi-AZ deployments with automatic failover and high fault tolerance.
- We are building microservices that benefit from distributed locks or shared memory constructs.

ElastiCache for Valkey

Valkey is a community-driven, Redis-compatible in-memory data store. It was created by the open-source community as a continuation of the Redis open-source project, offering Redis protocol compatibility with community-led innovation. Valkey supports all major Redis commands and data types and is ideal for teams seeking a more transparent development model.

Ask

Consider using Valkey in the following scenarios:

- We need Redis compatibility with open governance for the application.
- We need to run modern workloads that need fast in-memory access and replication.

ElastiCache for Memcached

Memcached is a high-performance, distributed memory caching system that is simple, fast, and effective for straightforward caching needs. It does not support complex data types, persistence, or replication, but excels in scenarios where lightweight caching is sufficient.

Consider using Memcached in the following scenarios:

- We need to implement the simplest model.
- We need the flexibility to add and remove nodes as the application scales.

The following table draws a comparison between the three supported caching engines for ElastiCache.

Feature	Redis	Valkey	
Persistence	Yes	Yes	
Multi-AZ Support	Yes	Yes	

Ask

Data Structures	Rich (e.g., lists, sets, and hashes)	Same as Redis OSS	
Clustering/Partitioning	Yes	Yes	
Use Case Complexity	High	High	

Backup and restore

Amazon ElastiCache offers daily automatic data backups. To set up automatic backups, we can configure the following settings:

- **Retention period:** This determines the number of days for which the backup is stored on the S3 bucket. The retention period can range from 0 to 35 days, where 0 days disables automatic backups.
- **Backup start time:** We can configure the time of day when the ElastiCache begins creating a backup of the data. We can set it up for the most convenient time of the day. If we do not set up the time, ElastiCache automatically assigns one.

We can also take manual snapshots of the cache and retain them for as long as we want. Data can be restored from both backups at any time by creating a new cluster from the backup. All backups are stored in Amazon S3 storage.

ElastiCache use cases

Most of the applications, whether it's in gaming, analytics, or e-commerce, rely heavily on speed. This speed can be achieved using effective caching strategies in applications. Developers should consider implementing caching mechanisms in their applications when:

Ask

- The data is slow or expensive to retrieve from the backend database.

- Users frequently access data to decrease infrastructure costs.
- The data is relatively static and does not change often.

Let's understand the efficacy of caching through some real-world applications.

Session management

Storing user session data in ElastiCache helps deliver faster login and session validation, which is essential for applications like e-commerce websites or content platforms. Redis is especially useful here due to its support for data expiration, eviction policies, and durability.

Pub/sub messaging

ElastiCache for Redis supports the pub/sub messaging pattern, allowing applications to instantly broadcast messages to multiple subscribers. This is especially useful for building chat systems, live dashboards, or collaborative tools.

For example, in a collaborative editing tool, when one user makes an update, a message can be published to a Redis channel. All clients subscribed to that channel receive the update in real time, enabling a seamless collaborative experience.

Valkey is a viable alternative to Redis, offering similar functionality with a stronger open-source guarantee and architectural improvements aimed at better performance and cost efficiency.

Gaming leaderboard microservice

Consider an online multiplayer game with millions of concurrent players. The game has a leaderboard displaying players' scores or ranks for a very short duration (e.g., the last 60 seconds of gameplay, or current in-match points) that needs to be updated and retrieved by hundreds of thousands of requests per second. The data is constantly changing and has an extremely short lifespan before it becomes irrelevant.

Ask

ElastiCache for Memcached is ideal for this scenario due to its unmatched simplicity and raw speed for basic key-value operations at extreme scale. The mini-leaderboard data is highly transient and ephemeral; if a cache node fails, the data for those few seconds is lost, but it's immediately regenerated by the game engine for the next second's update. There's no need for complex Redis data structures like sorted sets (the data is too short-lived for complex sorting operations within the cache), persistence, or other advanced features that add overhead. Memcached's multithreaded architecture efficiently utilizes multiple

Ask